© Guven

Multilayer Perceptron

Introduction

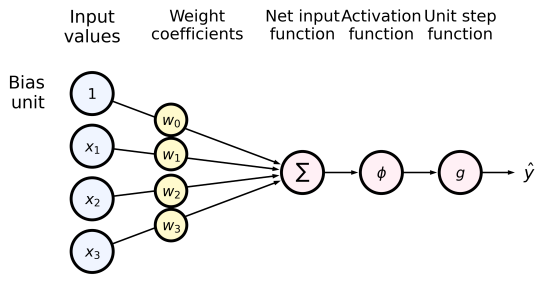
```
"Each problem that I solved became a rule which served afterwards to solve other problems." Descartes
```

Inspired by the **brain**, a neural network consists of **neurons** which operate similarly to their counterparts in the brain.

A neuron receives input from roughly thousands of other neurons via its **synapses**. These inputs are approximately summed, and a threshold function is applied to each. When the input exceeds this threshold, the neuron generates an electrical spike that travels down the __axon__to the next set of neurons.

A perceptron follows a similar kind of behavior in mathematical terms.

Single-layer Neural Network



The perceptron is a binary classifier that *attempts* to find an M dimensional hyperplane w defined by $w \cdot x = w^\top x = b$ where $x, w \in \mathbb{R}^M$, and $b \in \mathbb{R}$.
There are M dimensions/features in the dataset of N instances/rows.

The hyperplane separates the data points X into two categories {0,1}, originally provided by the y vector (ground truth).

$$\mathbf{f}(x) = \hat{y} = \begin{cases} 1, & \text{if } w \cdot x + b > 0 \\ 0, & \text{otherwise} \end{cases}$$

Finding out the hyperplane or w, the perceptron training algorithm, such as Adaline, updates w by iteratively computing the classification error:
$w \leftarrow w + \Delta w$, where $\Delta w = -\eta \nabla J(w)$ and $\nabla J(w)$ is the cost function and η is the learning rate.

The gradient of J, or $\nabla J(w)$ is computed by partial derivatives:
$\frac{\partial}{\partial w_j} J(w) = -\sum_i \left(y^{(i)} - a^{(i)}\right) x_j^{(i)}$, for $j = 1, \ldots, M$ and $i = 1, \ldots, N$

Activation function $\phi(z)$ where the net input z is the linear combination of the weights that are connecting the input to the output: $z = \sum_j w_j x_j = w^\top x$
The threshold function $g(z)$ is just a 0-threshold or binary step:

$$\hat{y} = \begin{cases} +1, & \text{if } g(z) \geq 0 \\ -1, & \text{otherwise} \end{cases}$$

Note that $g(\cdot)$ is needed in a perceptron, but in a neural network, it is not used.

XOR Problem

Since a perceptron uses a single hyperplane, it cannot solve the XOR problem because it is **not linearly separable.** An XOR problem is presented in the plot below, where two classes reside in opposing quadrants.

For example, the XOR problem in 2 dimensions is as follows. As can be seen from the classification, a single perceptron, i.e., linear SVM, cannot solve the problem.

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 72
import numpy as np
from numpy.random import ranf
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier

import warnings
warnings.filterwarnings('ignore', 'Solver terminated early.*')
warnings.filterwarnings('ignore', 'Stochastic Optimizer:*')

# Example random XOR dataset
X = np.concatenate([np.array([-1.5,-1.5]) + ranf((10,2)), np.array([0.5, 0.5]) + ranf((10,2)),
                    np.array([-1.5, 0.5]) + ranf((10,2)), np.array([0.5,-1.5]) + ranf((10,2))], axis=0)
y = np.concatenate([np.full(10, 0), np.full(10, 0), np.full(10, 1), np.full(10, 1)], axis=0)

X_mn, X_mx = -2, 2

fig, ax = plt.subplots(nrows=1, ncols=5, sharex='none', sharey='all', figsize=(14, 3), dpi=72)

# Plot the data points
def plotX(_ax, _X, _y, _title):
    _ax.scatter(_X[_y==0,0], _X[_y==0,1], marker='o', color='darkorchid', s=50)
    _ax.scatter(_X[_y==1,0], _X[_y==1,1], marker='+', color='darkgreen', s=90)
    _ax.set_xlim(X_mn, X_mx)
    _ax.set_ylim(X_mn, X_mx)
    _ax.set_title(_title)

# Plot the decision boundary of a classifier using mesh and contour, 2-dims
def plot_decisionboundary(_ax, _h, _X, _clf, color='r'):  # h = step size in the mesh
    xx1, xx2 = np.meshgrid(np.arange(X_mn, X_mx, _h), np.arange(X_mn, X_mx, _h))
    Y = _clf.predict(np.c_[xx1.ravel(), xx2.ravel()]).reshape(xx1.shape)
    _ax.contour(xx1, xx2, Y, colors=color, linestyles='dashed')

def svc_linear(_X, _y, _ax, C=10, color='r'):
    clf = SVC(kernel='linear', C=C, tol=1e-2).fit(_X, _y)
    # dividing hyperplane
    a = -clf.coef_[0][0] / clf.coef_[0][1]
    x1_plot = np.linspace(X_mn, X_mx)
    x2_plot = a*x1_plot - clf.intercept_[0]/clf.coef_[0][1]
    _ax.plot(x1_plot, x2_plot, linestyle='--', color=color, linewidth=2, label=f'C={C:g}')

def svc_rbf(_X, _y, _ax, _gamma, C=10, color='r'):
    clf = SVC(kernel='rbf', gamma=_gamma, C=C).fit(_X, _y)
    plot_decisionboundary(_ax, 0.01, _X, clf, color)

def nn_(_X, _y, _ax, _nsize, color='orange'):  # a single perceptron NN or single layer NN
    clf = MLPClassifier(hidden_layer_sizes=(_nsize,), max_iter=300, solver='lbfgs', random_state=0).fit(_X, _y)
    plot_decisionboundary(_ax, 0.01, _X, clf, color)

plotX(ax[0], X, y, 'Input random XOR data')

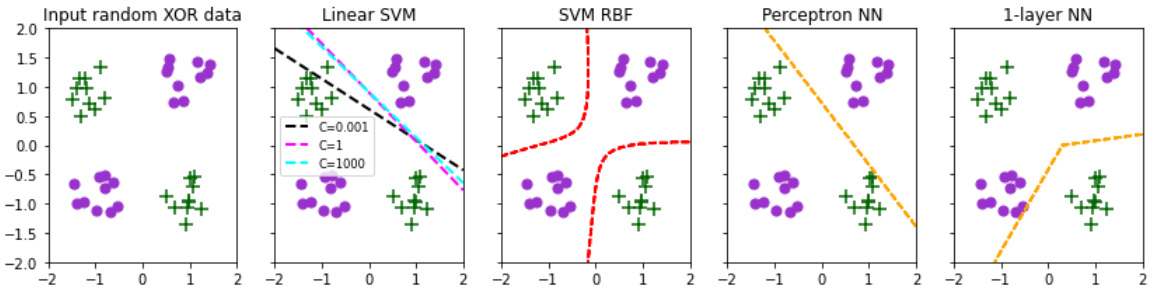
plotX(ax[1], X, y, 'Linear SVM')
svc_linear(X, y, ax[1], C=1e-3, color='k')
svc_linear(X, y, ax[1], C=1, color='magenta')
svc_linear(X, y, ax[1], C=1e3, color='cyan')
ax[1].legend(fontsize='small')

plotX(ax[2], X, y, 'SVM RBF')
svc_rbf(X, y, ax[2], 0.5)

plotX(ax[3], X, y, 'Perceptron NN')
nn_(X, y, ax[3], 1)
```

```
plotX(ax[4], X, y, '1-layer NN')
nn_(X, y, ax[4], 2)

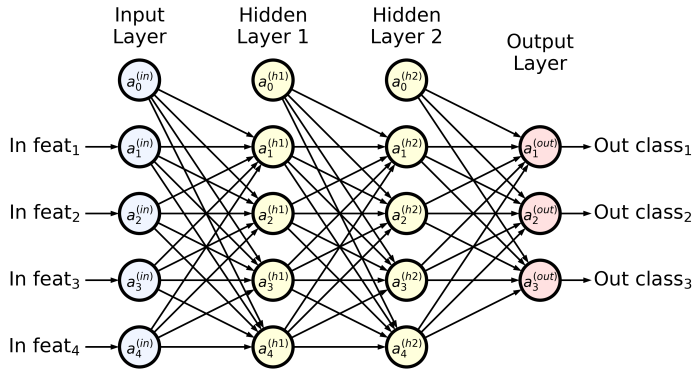
plt.show()
```



Question: Can you improve the last classifier above by a larger 1-layer?

Multilayer Neural Network Architecture

In a multilayer neural network structure, the input is weighted and summed at one or more hidden layers representing a directed graph. When each layer output is input to the next layer, this architecture is called **feedforward** neural network. When the output of a layer is **fed-back** to a previous layer then it is called **recurrent** neural network in which from a time step to the next, some neuron layers will remember information they had in the previous time-step. This gives the neuron the ability to work as a memory cell.



The input vector $a^{(in)}$ has a bias value: $a^{(in)} = \begin{bmatrix} 1 \\ x_1^{(in)} \\ \vdots \\ x_m^{(in)} \end{bmatrix}$

The output vector is one-hot encoded categories.

In the figure, an example feedforward neural network is provided with 2 hidden layers, M=4, and the number of classes is 3 and $a_0^{(in)}, a_0^{(h1)}, a_0^{(h2)}$ are bias units and set to 1. The predicted class can be stated by checking the highest value on the output layer, e.g., if $a_2^{(out)}$ is the highest, then the predicted class is 2.

Multilayer feedforward architectures are universal function approximators.

Deep Neural Networks: When there is more than 1 hidden layer, the neural network is called a **deep** neural network. Note that a convolutional neural network as an image classifier can have thousands of layers in practice.

Training MLP via Forward Propagation

The training data is propagated on the network, and based on the output error, network weights are updated to minimize the cost function. The error at the output is **backpropagated**, and the model has been updated. (Note that derivative is a linear operator)

Net input, $z_1^{(h)} = a_0^{(in)} w_{0,1}^{(h)} + a_1^{(in)} w_{1,1}^{(h)} + .. + a_m^{(in)} w_{m,1}^{(h)}$

Activation, $a_1^{(h)} = \phi\left(z_1^{(h)}\right)$

The activation function has to be differentiable so we can use gradient-based approaches to minimize the error and find weights. Generally, non-linear activation functions work well and one example is the sigmoid function:

$\phi(z) = \frac{1}{1+e^{-z}}$ ..... $(Eq.1)$

Clearly `numpy` uses efficient loops for linear algebra operations. Net input and activation for all N data points:

$Z^{(h)} = A^{(in)} W^{(h)}$, then $A^{(h)} = \phi\left(Z^{(h)}\right)$, and finally $Z^{(out)} = A^{(h)} W^{(out)}$, and $A^{(out)} = \phi\left(Z^{(out)}\right)$ ..... $(Eq.2)$

Sigmoid function derivative:

$\phi' = \frac{\partial}{\partial z}\left(\frac{1}{1+e^{-z}}\right) = \phi(z)(1-\phi(z)) = a(1-a)$ ..... $(Eq.3)$

Cost function:

$J(w) = \frac{1}{2}\sum_i \left(y^{[i]} - \phi\left(z^{[i]}\right)\right)^2$ i.e. Sum of Squared Errors

Derivative of cost function:

$\frac{\partial J}{\partial w_j} = -\sum_i \left(y^{[i]} - \phi\left(z^{[i]}\right)\right) x_j^{[i]}$

Log-likelihood cost function:

$J(w) = -\sum_{i=1}^{n} \left[y^{[i]} \log\left(\phi\left(z^{[i]}\right)\right) + \left(1-y^{[i]}\right) \log\left(1-\phi\left(z^{[i]}\right)\right)\right]$ since maximizing log of the cost is easier and does not break the optimization.

Substitute a, the main cost function for optimizing the neural network (without the regularization component):

$J(w) = -\sum_{i=1}^{n} y^{[i]} \log\left(a^{[i]}\right) + \left(1-y^{[i]}\right) \log\left(1-a^{[i]}\right)$ ..... $(Eq.4)$

Backpropagate the error from the output to the input:

$\delta^{(out)} = a^{(out)} - y$ ..... $(Eq.5)$

$\delta^{(h)} = \delta^{(out)} \left(W^{(out)}\right)^\top \odot \left(a^{(h)} \odot \left(1-a^{(h)}\right)\right)$ ..... $(Eq.6)$

$\frac{\partial}{\partial w_{i,j}^{(out)}} J(W) = a_j^{(h)} \delta_i^{(out)}$ ..... $(Eq.7)$

$\frac{\partial}{\partial w_{i,j}^{(h)}} J(W) = a_j^{(in)} \delta_i^{(h)}$ ..... $(Eq.8)$

Update weights:

$W^{(l)} = W^{(l)} - \eta \Delta^{(l)}$ ..... $(Eq.9)$

Derivation of $Eq.4$ to $Eq.7$:

Given:

1. $a^o = a^h w^o \implies \frac{\partial a^o}{\partial w^o} = a^h$
2. $\phi^{out\prime} = a^o(1-a^o)$

Then $\frac{\partial J}{\partial w^o} = y\frac{1}{a^o}\frac{\partial a^o}{\partial w^o}\phi^{out\prime} + (1-y)\frac{1}{1-a^o}(-\frac{\partial a^o}{\partial w^o})\phi^{out\prime} = ya^h(1-a^o) + (1-y)(-a^h)(a^o) = (y-a^o)a^h = \delta^o a^h$

Worked Example

The following example uses MNIST dataset: MNIST database

http://yann.lecun.com/exdb/mnist/

In [2]:
```
def load_mnist(path, kind='train'):
    from numpy import fromfile, uint8
    import os
    import struct

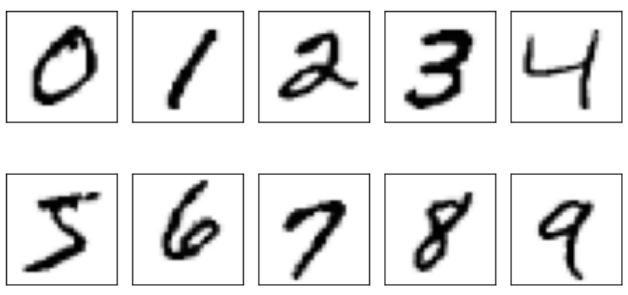
    labels_path = os.path.join(path, '%s-labels-idx1-ubyte' % kind)
    images_path = os.path.join(path, '%s-images-idx3-ubyte' % kind)
    with open(labels_path, 'rb') as lbpath:
        magic, n = struct.unpack('>II', lbpath.read(8))
        labels = fromfile(lbpath, dtype=uint8)
        with open(images_path, 'rb') as imgpath:
            magic, num, rows, cols = struct.unpack(">IIII",imgpath.read(16))
            images = fromfile(imgpath, dtype=uint8).reshape(len(labels), 784)
            images = ((images / 255.) - .5) * 2
    return images, labels

X_train_mnist, y_train_mnist = load_mnist('/EP_datasets/mnist/', kind='train')
print(f'Rows= {X_train_mnist.shape[0]}, columns= {X_train_mnist.shape[1]}')

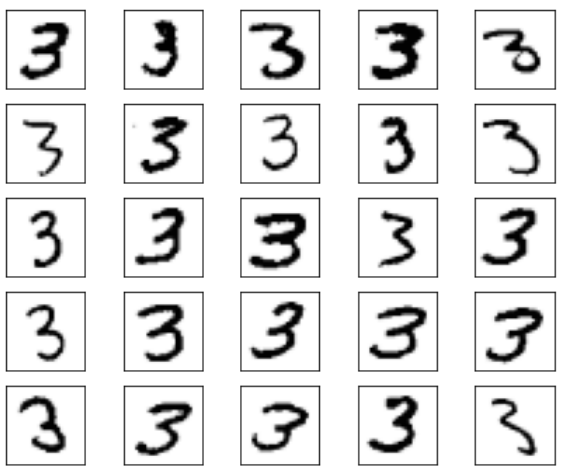
X_test_mnist, y_test_mnist = load_mnist('/EP_datasets/mnist/', kind='t10k')
print(f'Rows= {X_test_mnist.shape[0]}, columns= {X_test_mnist.shape[1]}')
```

```
Rows= 60000, columns= 784
Rows= 10000, columns= 784
```

In [3]:
```
_, ax = plt.subplots(nrows=2, ncols=5, sharex='all', sharey='all', dpi=72)
ax = ax.flatten()
for img_i in range(10):
    img = X_train_mnist[y_train_mnist == img_i][0].reshape(28, 28)
    ax[img_i].imshow(img, cmap='Greys')
    ax[0].set_xticks([])
    ax[0].set_yticks([])
    plt.tight_layout()
plt.show()
```



In [4]:
```
fig, ax = plt.subplots(nrows=5, ncols=5, sharex='all', sharey='all', dpi=72)
ax = ax.flatten()
for img_i in range(25):
    img = X_train_mnist[y_train_mnist == 3][img_i].reshape(28, 28)
    ax[img_i].imshow(img, cmap='Greys')
    ax[0].set_xticks([])
    ax[0].set_yticks([])
    plt.tight_layout()
plt.show()
```



In [5]:
```
from sklearn.metrics import confusion_matrix

def get_acc(_y_test, _y_pred):
    return (np.sum(_y_test == _y_pred)).astype(float) / _y_test.shape[0]
```

In [6]:
```
%%time

# 1 layer neural network
mlp = MLPClassifier(hidden_layer_sizes=(10,), max_iter=200, activation='logistic', random_state=0)
mlp.fit(X_train_mnist, y_train_mnist)

y_pred = mlp.predict(X_test_mnist)

print(f'Accuracy= {get_acc(y_test_mnist, y_pred)*100:.2f}%')
print(confusion_matrix(y_test_mnist, y_pred))
```

```
Accuracy= 93.22%
[[ 962    0    4    0    0    7    4    1    2    0]
 [   0 1105   10    2    0    0    4    0   13    1]
 [   9    9  957    7    7    2    9    5   22    5]
 [   3    3   22  931    0   21    2    8   17    3]
 [   2    2    7    0  917    0   13    1    5   35]
 [   9    3    5   30    6  788   14    3   25    9]
 [  16    1   13    0    5    9  909    0    5    0]
 [   2    9   25    5    7    0    0  958    6   16]
 [   8    5   12   19    9   18    7   14  878    4]
 [   5    6    0   11   28   15    0   22    5  917]]
CPU times: total: 1min 23s
Wall time: 37.9 s
```

In [7]:
```
%%time

# 2 layer neural network
mlp = MLPClassifier(hidden_layer_sizes=(10,10),max_iter=200, random_state=0)
mlp.fit(X_train_mnist, y_train_mnist)

y_pred = mlp.predict(X_test_mnist)
```

```
print(f'Accuracy= {get_acc(y_test_mnist, y_pred)*100:.2f}%')
print(confusion_matrix(y_test_mnist, y_pred))
```

```
Accuracy= 93.49%
[[ 958    0    2    2    2    3    6    5    1    1]
 [   0 1105    3    4    1    2    3    3   14    0]
 [  11    1  960   18    6    3    8   10   15    0]
 [   5    0   19  924    2   22    1   13   24    0]
 [   2    0    8    0  930    2    8    2    3   27]
 [  13    4    2   19    4  800   15    4   21   10]
 [  16    2    9    1   10   17  891    0   12    0]
 [   2    5   16   13    8    0    0  960    2   22]
 [   4    3    3   23    7   13   14    4  891   12]
 [   8    4    0    5   35    7    0   12    8  930]]
CPU times: total: 10min 4s
Wall time: 1min 11s
```

In [8]:
```
%%time

# 1 layer neural network, size 20
mlp = MLPClassifier(hidden_layer_sizes=(20,), max_iter=200, random_state=0)
mlp.fit(X_train_mnist, y_train_mnist)

y_pred = mlp.predict(X_test_mnist)

print(f'Accuracy= {get_acc(y_test_mnist, y_pred)*100:.2f}%')
print(confusion_matrix(y_test_mnist, y_pred))
```

```
Accuracy= 94.34%
[[ 956    0    3    1    2    6    3    4    2    3]
 [   0 1122    2    1    0    1    4    1    4    0]
 [   9    6  968   16    5    3    4   11   10    0]
 [   0    2   11  957    1    6    2   14   10    7]
 [   2    1    4    0  923    4   10    6    3   29]
 [  14    2    2   25    1  821    9    8    6    4]
 [  14    4    4    0    6   24  901    1    4    0]
 [   1    6   11    8    3    0    0  981    1   17]
 [   6    8    3   33   10   14    6   13  868   13]
 [   7    6    4   12   21    4    0   18    0  937]]
CPU times: total: 12min 39s
Wall time: 1min 21s
```

Question: Which network is better?
(A.) 1 hidden-layer, size 10
(B.) 2 hidden-layers, size 10 each
(C.) 1 hidden-layer, size 20?

Let's see a neural network with 1 hidden layer implemented from scratch. Note that the source lines that correspond to the above equations are commented.

In [9]:
```
class NeuralNetMLP(object):

    def __init__(self, n_hidden=30, epochs=100, eta=0.001, minibatch_size=1, seed=None):
        self.random = np.random.RandomState(seed)  # used to randomize weights
        self.n_hidden = n_hidden  # size of the hidden layer
        self.epochs = epochs  # number of iterations
        self.eta = eta  # learning rate
        self.minibatch_size = minibatch_size  # size of training batch - 1 would not work
        self.w_out, self.w_h = None, None

    @staticmethod
    def onehot(_y, _n_classes):  # one hot encode the input class y
        onehot = np.zeros((_n_classes, _y.shape[0]))
        for idx, val in enumerate(_y.astype(int)):
            onehot[val, idx] = 1.0
        return onehot.T

    @staticmethod
    def sigmoid(_z):  # Eq 1
        return 1.0 / (1.0 + np.exp(-np.clip(_z, -250, 250)))

    def _forward(self, _X):  # Eq 2
        z_h = np.dot(_X, self.w_h)
        a_h = self.sigmoid(z_h)
        z_out = np.dot(a_h, self.w_out)
        a_out = self.sigmoid(z_out)
        return z_h, a_h, z_out, a_out

    @staticmethod
    def compute_cost(y_enc, output):  # Eq 4
        term1 = -y_enc * (np.log(output))
        term2 = (1.0-y_enc) * np.log(1.0-output)
        cost = np.sum(term1 - term2)
        return cost

    def predict(self, _X):
        z_h, a_h, z_out, a_out = self._forward(_X)
        ypred = np.argmax(z_out, axis=1)
        return ypred

    def fit(self, _X_train, _y_train, _X_valid, _y_valid):
        import sys
        n_output = np.unique(_y_train).shape[0]  # number of class labels
        n_features = _X_train.shape[1]
        self.w_out = self.random.normal(loc=0.0, scale=0.1, size=(self.n_hidden, n_output))
        self.w_h = self.random.normal(loc=0.0, scale=0.1, size=(n_features, self.n_hidden))
        y_train_enc = self.onehot(_y_train, n_output)  # one-hot encode original y
        for ei in range(self.epochs):  # Ideally must shuffle at every epoch
            indices = np.arange(_X_train.shape[0])
            for start_idx in range(0, indices.shape[0] - self.minibatch_size + 1, self.minibatch_size):
                batch_idx = indices[start_idx:start_idx + self.minibatch_size]

                z_h, a_h, z_out, a_out = self._forward(_X_train[batch_idx])  # neural network model

                sigmoid_derivative_h = a_h * (1.0-a_h)  # Eq 3
                delta_out = a_out - y_train_enc[batch_idx]  # Eq 5
                delta_h = (np.dot(delta_out, self.w_out.T) * sigmoid_derivative_h)  # Eq 6
                grad_w_out = np.dot(a_h.T, delta_out)  # Eq 7
                grad_w_h = np.dot(_X_train[batch_idx].T, delta_h)  # Eq 8
                self.w_out -= self.eta*grad_w_out  # Eq 9
                self.w_h -= self.eta*grad_w_h  # Eq 9

            # Evaluation after each epoch during training
            z_h, a_h, z_out, a_out = self._forward(_X_train)
            cost = self.compute_cost(y_enc=y_train_enc, output=a_out)
            y_train_pred = self.predict(_X_train)  # monitoring training progress through reclassification
            y_valid_pred = self.predict(_X_valid)  # monitoring training progress through validation
            train_acc = ((np.sum(_y_train == y_train_pred)).astype(float) / _X_train.shape[0])
            valid_acc = ((np.sum(_y_valid == y_valid_pred)).astype(float) / _X_valid.shape[0])
            sys.stderr.write('\r%d/%d | Cost: %.2f ' '| Train/Valid Acc.: %.2f%%/%.2f%% '%
                (ei+1, self.epochs, cost, train_acc*100, valid_acc*100))
            sys.stderr.flush()
        return self
```

In [10]:
```
# Define and fit the neural network
nn = NeuralNetMLP(n_hidden=20, epochs=200, eta=0.0005, minibatch_size=100, seed=1)
```

```
nn.fit(X_train_mnist[:55000], y_train_mnist[:55000], X_train_mnist[55000:], y_train_mnist[55000:]) ;
```

```
200/200 | Cost: 16894.16 | Train/Valid Acc.: 95.75%/95.56%
```

In [11]:
```
y_pred = nn.predict(X_test_mnist)

print(f'Accuracy= {get_acc(y_test_mnist, y_pred)*100:.2f}%')
print(confusion_matrix(y_test_mnist, y_pred))
```

```
Accuracy= 94.20%
[[ 961    0    1    0    2    2    7    1    5    1]
 [   0 1099    4    3    1    0    3    1   24    0]
 [  11    0  962   14    4    1    8   12   20    0]
 [   3    3   14  953    0   16    1    7    9    4]
 [   0    0    2    1  939    1    5    4    5   25]
 [   8    3    2   38    7  785   11    7   23    8]
 [  14    3    5    0    8   16  900    1   11    0]
 [   3    6   15    4    2    0    0  985    3   10]
 [   5    4    4   14   10    9    4   10  910    4]
 [   4    5    0   17   24    6    1   21    5  926]]
```

Note that in the above code, the following are skipped:

- L2 regularization term (to diminish overfitting)
- Backpropagation components (to shape the sigmoid function)
- Shuffling of the input data (avoiding certain dataset orders that would lead to biased or slow network training)

The above code demonstrates the absolute primitive complete version of a 1 hidden layer neural network model.

Question: Do multiple runs lead to a variation in classification errors?

A Note on Data Structures

We often require shaping tensors using the `numpy.reshape()` or `torch.view()`. It is essential to note the row-major or column-major ordering:

- Python, PyTorch, Numpy, C++ languages, and compilers/interpreters have row-major order
- Matlab and Fortran have column-order

Example

In [12]:
```
# row-major order
A = np.array([[1,2,3,4],[5,6,7,8]])
print(A)
print(A.reshape(4,-1))
print(A.reshape(-1,2))
```

```
[[1 2 3 4]
 [5 6 7 8]]
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

References

1. Raschka, Sebastian, et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. Packt Publishing Ltd, 2022.

Exercises

Exercise 1. Place a few `print` statements in the neural network code to display the shape of the vectors `w_out`, `w_h`, `y_train_enc`, `z_h`, `a_h`, `z_out`, `a_out`, `y_pred`. Do you find them as expected? (Hint: Use PyCharm to debug)

Exercise 2. Plot the sigmoid function and verify that it maps: $[-\infty, +\infty] \rightarrow [0, 1]$. Which other function does sigmoid resemble?

In [13]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```
